

Lab 2: Traps 实验报告

李睿阳 524120910059

1. Backtrace

实现思路

- **原理**: 利用 `s0` 寄存器中维护的帧指针。栈帧布局固定: 返回地址在 `fp-8`, 前序帧指针在 `fp-16`。
- **过程**: 调用 `r_fp()` 获取当前 `fp`, 循环向上回溯并打印返回地址。

```
uint64 fp = r_fp();
while (fp < PGROUNDUP(fp) && fp > PGROUNDDOWN(fp)) {
    uint64 ra = *(uint64*)(fp - 8);
    printf("%p\n", ra);
    fp = *(uint64*)(fp - 16);
}
```

- **终止条件**: 由于 xv6 为每个进程分配一个页面的内核栈, 使用 `PGROUNDDOWN(fp)` 判断是否仍在当前栈页内。

修改文件

- `kernel/riscv.h`: 定义 `r_fp()` 获取 `s0`。
- `kernel/defs.h`: 声明 `backtrace()`。
- `kernel/printf.c`: 实现 `backtrace()`, 并在 `panic()` 中调用。
- `kernel/sysproc.c`: 在 `sys_sleep` 中插入 `backtrace()` 进行测试。

2. Alarm

实现思路

- **注册**: `sigalarm(n, handler)` 将间隔时间 `n` 和处理函数地址存入 `struct proc`。

```
uint64
sys_sigalarm(void)
{
    int n;
    uint64 handler;
    if(argint(0, &n) < 0 || argaddr(1, &handler) < 0)
        return -1;
    struct proc *p = myproc();
    p->alarm_interval = n;
    p->alarm_handler = handler;
    p->ticks_count = 0;
```

```
return 0;
}
```

- **触发**: 在 `usertrap()` 的时钟中断逻辑中增加计数。计数达标且无闹钟运行时, 备份现场并重定向 `epc`。

```
if (which_dev == 2) { // 时钟中断
    if (p->alarm_interval != 0 && p->alarm_running == 0) {
        p->ticks_count++;
        if (p->ticks_count >= p->alarm_interval) {
            memmove(p->alarm_trapframe, p->trapframe, sizeof(struct trapframe));
            p->trapframe->epc = p->alarm_handler;
            p->alarm_running = 1;
            p->ticks_count = 0;
        }
    }
}
```

- **恢复**: 用户 handler 调用 `sigreturn()`, 内核恢复备份现场。

```
uint64 sys_sigreturn(void) {
    struct proc *p = myproc();
    memmove(p->trapframe, p->alarm_trapframe, sizeof(struct trapframe));
    p->alarm_running = 0;
    return p->trapframe->a0;
}
```

修改文件

- **Makefile**: 添加 `_alarmtest`。
- **kernel/proc.h**: `struct proc` 增加 `interval`, `handler`, `ticks`, `running` 及 `alarm_trapframe` 备份指针。
- **kernel/proc.c**: `allocproc` 分配备份页, `freeproc` 释放。
- **user/user.h**: 声明 `sigalarm` 和 `sigreturn`。
- **user/usys.pl**, **kernel/syscall.h**, **kernel/syscall.c**: 注册系统调用。
- **kernel/sysproc.c**: 实现 `sys_sigalarm` 和 `sys_sigreturn`。
- **kernel/trap.c**: `usertrap` 中间逻辑实现触发跳转。

3. 调试过程中遇到的问题

- 内核函数只接受 `void`: `sys_sigalarm` 等系统调用封装函数在 C 语言层面不接受参数, 必须通过 `argint(0, &n)` 或 `argaddr(1, &addr)` 获取用户传入的值。
- `sigreturn` 要返回 `a0` 来保持原状态: `sigreturn` 是一个系统调用, 其正常返回会覆盖 `trapframe->a0`。必须在 `sys_sigreturn` 中显式返回恢复后的 `a0` 原值, 否则返回后进程的寄存器状态会被破坏。
- 在 `sys_sigreturn` 不要忘记把 `p->alarm_running` 置 0, 否则此后的闹钟将永远无法再次触发。

- 不要忘记在 `user/user.h` 中添加用户态函数声明：

```
int sigalarm(int ticks, void (*handler)());  
int sigreturn(void);
```

- 在 `sigreturn` 中要用 `memmove` 把备份的结构体内容搬回 `trapframe`，复制指针地址是不严谨的，会导致后续的问题。
- 在 `allocproc` 中要用 `kalloc` 为 `alarm_trapframe` 分配一个物理页，并在 `freeproc` 中使用 `kfree` 释放，避免内存泄漏。